Projet 1 : Système de gestion de tournois sportifs

Présentation générale du projet

Le projet consiste à développer une application permettant de gérer facilement des tournois sportifs, quels que soient le sport et le format du tournoi. Le système devra automatiser la création des tournois, la planification des matchs, le classement et l'application des règles selon le type de compétition.

L'application vise principalement les organisateurs de tournois (clubs, associations, écoles, entreprises), afin de simplifier la logistique et d'éviter les erreurs manuelles.

Objectifs du projet

- Faciliter l'organisation de tournois grâce à un outil intuitif.
- Automatiser les tâches répétitives : génération du calendrier, classement, matchs, gestion des égalités.
- Supporter plusieurs types de tournois :
  - Élimination directe (KO)
  - Championnat (round-robin)
- Offrir un tableau de bord clair pour suivre l'évolution du tournoi.

Fonctionnalités attendues

Création de tournois

- Création d'un tournoi avec : nom, sport, type de tournoi, lieu, date début/fin.
- Configuration des paramètres : nombre d'équipes, durée des matchs, terrains disponibles.
- Possibilité d'ajouter des règles spécifiques.

Gestion des équipes

- Ajout, modification et suppression d'équipes.
- Fiche équipe : nom, logo, contact.
- Affectation automatique ou manuelle dans le tableau du tournoi.

Génération automatique du calendrier

- Création automatique du calendrier selon :
  - type de tournoi
  - nombre d'équipes
  - disponibilité des terrains
- Possibilité de réorganiser manuellement les matchs.

Gestion des matchs

- Génération automatique des matchs (phase de poules, quart, demi, finale…).
- Saisie des scores.

- Mise à jour automatique du classement ou du bracket (arbre de tournoi).
- Gestion des prolongations, tirs au but (optionnel selon sport).

Classement automatique

Selon le type de tournoi :

Championnat (round-robin)

- Calcul des points (victoire, nul, défaite).
- Gestion des égalités :
  - différence de buts
  - confrontation directe
  - nombre de buts marqués.
- Classement mis à jour en temps réel.

Élimination directe

- Bracket généré automatiquement.
- Qualification automatique du vainqueur au tour suivant.

Règles de tournoi

L'application doit gérer plusieurs formats :

Élimination directe

- Arbre automatique.
- Nombre de tours calculé automatiquement en fonction du nombre d'équipes.

Championnat

- Matches aller ou aller-retour.
- Points configurables (ex : 3 points victoire / 1 point nul).

Logique métier (Business Logic)

Calcul automatique du classement

Le système doit :

- mettre à jour le nombre de points après chaque match ;
- recalculer le classement dès qu'un score est modifié ;
- appliquer les critères de départage ;
- assurer la cohérence des statistiques (buts marqués, encaissés, différence).

Gestion des égalités

Lors d'égalité de points :

(1)Différence de buts

(2)Confrontation directe

(3)Nombre de buts marqués

(4)Nombre de buts encaissés

(5)Tirage au sort (en dernier recours)

Génération automatique des matchs

Le système doit être capable de :

- générer les rencontres selon l'algorithme adapté :
  - round-robin pour un championnat
  - pairing KO pour élimination directe
- ajuster les horaires et terrains en fonction des contraintes.

Contraintes techniques

- **Backend** : Java EE.
- **Frontend** : Au choix, mais responsive.
- **Base de données** : MySQL ou PostgreSQL.
- API REST pour communication.

Interfaces attendues

Interface administrateur / organisateur

- Gestion des tournois
- Gestion des équipes
- Saisie des résultats
- Vue du bracket ou du classement
- Écran de configuration des règles

Interface publique

- Visualisation des matchs
- Classement en direct
- Résultats
- Bracket visuel

Sécurité et performance

- Authentification sécurisée
- Rôles : Admin, Organisateur, Spectateur
- Validation des données
- Protection contre les incohérences de score

ESMIA

Projet 2 : Système de gestion de commandes dans une entreprise textile

Présentation générale du projet

L'objectif du projet est de développer un système permettant de centraliser et automatiser la gestion des commandes clients dans une entreprise textile.
L'outil doit assurer le suivi détaillé du processus de production textile, depuis la réception de la commande jusqu'à la livraison finale.

Le système doit également prendre en charge le workflow de production, le calcul automatique des délais et le blocage d'avancement si une étape n'est pas terminée.

Objectifs du projet

- Améliorer l'efficacité de gestion des commandes.
- Offrir une visibilité complète sur l'état d'avancement de la production.
- Réduire les erreurs humaines grâce à un workflow automatisé.
- Optimiser les délais de livraison.
- Assurer un suivi strict des étapes : coupe → couture → finition → livraison.

Fonctionnalités attendues

Gestion des commandes clients

- Ajout, modification et suppression des commandes.
- Informations requises :
  - Nom du client
  - Type d'article / modèle
  - Quantité
  - Taille / couleur (si nécessaire)
  - Date prévue de livraison
- Statut de commande : En attente, En production, Terminé, Livré.

Suivi de production

- Affichage en temps réel de l'état de chaque commande.
- Historique des actions pour chaque commande.
- Visualisation globale des commandes en production.

Gestion des étapes de production

Chaque commande passe obligatoirement par les étapes suivantes :

(1)Coupe

(2)Couture

(3)Finition

(4)Livraison

Pour chaque étape :

- Début / fin
- Responsable de l'étape
- Validation de l’étape

Logique métier (Business Logic)

Workflow de production

Le système doit imposer un ordre strict des étapes. Une étape ne peut commencer que si la précédente est validée.

Exemple :

- Pas de **couture** tant que la **coupe** n’est pas terminée.
- Pas de **livraison** tant que la **finition** n’est pas validée.

Le workflow peut être représenté comme une chaîne bloquante :

Coupe → Couture → Finition → Livraison

Calcul des délais

Le système doit :

- calculer automatiquement les délais de production par commande ;
- indiquer la date estimée de fin de chaque étape ;
- signaler les retards potentiels (si une étape dépasse son délai prévu).

Le délai global est calculé en fonction :

- des durées prévues pour chaque étape ;
- de la date de début réelle des travaux ;
- des retards enregistrés.

Blocage d’avancement

Si une étape n’est pas terminée, le système doit bloquer :

- la validation de l’étape suivante ;
- la mise à jour du statut global ;
- la livraison de la commande.

Ce blocage doit être clairement notifié aux utilisateurs, avec un message d’erreur explicite.

Contraintes techniques

- Backend : Java EE.
- Frontend : Au choix, mais responsive.
- Base de données : MySQL ou PostgreSQL.

Interfaces attendues

Interface Administrateur

- Gestion des utilisateurs
- Consultation de toutes les commandes
- Gestion des étapes et des délais
- Tableau de bord global

Interface Opérateur de production

- Voir les commandes assignées
- Mettre à jour l'état des étapes
- Ajouter des remarques / problèmes rencontrés

Interface Client

- Consultation de l'avancement
- Historique des commandes
- Notifications de livraison

Sécurité et gestion des accès

- Système d'authentification sécurisé
- Rôles : Admin, Responsable Production, Opérateur, Client

ESMIA

Projet 3 : Application de gestion de flotte de taxis

Présentation générale

L'objectif du projet est de créer une application permettant de gérer efficacement une flotte de taxis : chauffeurs, véhicules, courses et revenus.

L'outil doit automatiser la distribution des courses, suivre la disponibilité des chauffeurs et fournir des statistiques sur l'activité globale afin d'optimiser la rentabilité de l'entreprise.

Objectifs du projet

- Centraliser les informations sur les chauffeurs et les véhicules.
- Assurer une attribution rapide et automatique des courses selon la disponibilité.
- Calculer précisément les revenus générés.
- Fournir des indicateurs utiles pour les décisions de gestion (statistiques d'activité).
- Gérer en temps réel l'état de fonctionnement de la flotte.

Fonctionnalités attendues

Gestion des chauffeurs

- Ajout, modification, suppression des chauffeurs.
- Informations enregistrées :
  - Identité du chauffeur
  - Numéro de téléphone
  - Permis de conduire
  - Statut (Disponible / En course / En pause / Indisponible)
  - Historique des courses effectuées
  - Revenus générés

Gestion des véhicules

- Ajout, modification, suppression des véhicules.
- Informations enregistrées :
  - Marque / modèle
  - Immatriculation
  - Kilométrage
  - Statut (Disponible / En mission / En maintenance)
  - Chauffeur affecté (optionnel ou automatique)

Attribution automatique des courses

- Le système doit automatiser l'affectation d'une course selon :
  - la disponibilité du chauffeur
  - la proximité géographique (si géolocalisation activée)

    - l'ordre d'arrivée des demandes
    - la charge de travail du chauffeur (répartition équitable)
- Pour les cas particuliers :
    - affectation manuelle en cas de besoin
    - réassignation si le chauffeur refuse ou n'est plus disponible

Gestion des courses

- Demande de course
- Attribution automatique ou manuelle
- Début / fin de la course

Calcul tarifaire

- Historique complet des courses
- Statut en temps réel : En attente / Assignée / En cours / Terminée / Annulée

Logique métier

Calcul des revenus

- Le système doit calculer automatiquement les revenus selon les paramètres suivants :
    - Tarif au kilomètre
    - Temps d'attente
    - Frais supplémentaires (bagages, nuit, etc.)
- Commission entreprise / chauffeur
    - Le revenu d'une course peut être calculé par exemple via :
    - Revenu chauffeur = Total course – Commission
    - Revenu entreprise = Commission
- Les rapports de revenus doivent être disponibles :
    - par chauffeur
    - par véhicule
    - par période (jour, semaine, mois)

Disponibilité des chauffeurs

- Un chauffeur peut être dans l'un des statuts suivants :
    - Disponible
    - En course
    - En pause
    - Indisponible
- Règles importantes :
    - Un chauffeur indisponible ne peut pas être affecté à une course.

- Un chauffeur en pause ne peut être assigné qu'après validation.
- Au début d'une course → devient "En course".
- À la fin → redevient "Disponible".

Statistiques d'activité

- Le système doit fournir des statistiques telles que :
  - Nombre de courses par chauffeur
  - Revenus par chauffeur / véhicule / jour
  - Courses annulées vs réussies
  - Temps moyen d'attente d'un client
  - Taux d'utilisation des taxis
  - Distance totale parcourue
- Ces statistiques doivent être visualisables sous forme :
  - de tableaux
  - de graphiques
  - ou de rapports imprimables

Contraintes techniques

- Backend : Java EE
- Frontend : Au choix, mais responsive
- Base de données : MySQL ou PostgreSQL
- API REST pour communication entre modules

Interfaces attendues

Interface Administrateur

- Gestion chauffeurs
- Gestion véhicules
- Configuration des paramètres (tarifs, commissions)
- Consultation des statistiques
- Historique global des courses

Interface Opérateur (dispatch)

- Création des commandes de course
- Visualisation en temps réel des chauffeurs disponibles
- Réassignation si nécessaire

Interface Chauffeur

- Réception de la course
- Acceptation / refus

- Navigation GPS (si intégré)
- Début et fin de course
- Bilan quotidien des revenus

Interface Client

- Demande de taxi
- Suivi du chauffeur en temps réel
- Paiement (simulation)
- Historique des courses

Sécurité et Permissions

- Authentification et rôles : Admin / Opérateur / Chauffeur
- Accès limité par rôle

ESMIA

Projet 4 : Système de gestion d'un parking

Présentation générale

Le projet consiste à développer une application permettant de gérer efficacement un parking, en automatisant la réservation des places, la gestion des abonnements et le suivi des entrées et sorties des véhicules.

Le système devra optimiser l'utilisation des places disponibles, calculer automatiquement les tarifs et gérer des priorités d'accès pour certains types d'utilisateurs (abonnés, VIP, etc.).

Objectifs du projet

- Automatiser la gestion des places de parking.
- Réduire les erreurs liées à la gestion manuelle.
- Optimiser le taux d'occupation du parking.
- Offrir un système de réservation simple et rapide.
- Gérer différents types d'utilisateurs avec priorités.
- Calculer automatiquement les frais de stationnement.

Fonctionnalités attendues

Réservation de places

- Réservation d'une place à l'avance.
- Choix de la date et de la durée.
- Attribution automatique d'une place disponible.
- Annulation ou modification de réservation.

Gestion des abonnements

- Création et gestion d'abonnements (mensuel, hebdomadaire, annuel).
- Association d'un utilisateur à un abonnement.
- Accès prioritaire pour les abonnés.
- Suivi de validité (date de début et de fin).

Suivi des entrées/sorties

- Enregistrement des entrées (heure d'arrivée).
- Enregistrement des sorties (heure de départ).
- Identification du véhicule (numéro de plaque).
- Historique des passages.

Logique métier

Calcul automatique du tarif

Le système doit calculer automatiquement le coût du stationnement en fonction de :

- la durée de stationnement ;

- le type d’utilisateur (abonné ou non) ;
- les règles tarifaires (heure, forfait, etc.).

Exemple :

- Tarif horaire : X Ar/heure
- Tarif journalier : forfait
- Gratuité ou réduction pour abonnés

Gestion des places disponibles

Le système doit :

- connaître en temps réel le nombre de places disponibles ;
- attribuer automatiquement une place libre ;
- empêcher toute réservation ou entrée si le parking est complet ;
- libérer automatiquement une place lors de la sortie d’un véhicule.

Priorités pour certains utilisateurs

Le système doit gérer des niveaux de priorité :

Types d’utilisateurs :

- Abonnés
- VIP
- Clients occasionnels

Règles :

- Les abonnés peuvent réserver à l’avance.
- Les VIP peuvent accéder même en cas de forte occupation (quota réservé).
- Les utilisateurs standards dépendent des places restantes.

Contraintes techniques

- Backend : Java EE
- Frontend : Au choix, mais responsive
- Base de données : MySQL ou PostgreSQL
- API REST

Interfaces attendues

Interface Administrateur

- Gestion des places
- Gestion des abonnements
- Configuration des tarifs
- Consultation des statistiques
- Historique global

Interface Agent de parking

- Enregistrement des entrées/sorties
- Vérification des réservations
- Attribution manuelle si nécessaire

Interface Client

- Réservation de place
- Consultation des abonnements
- Historique des stationnements
- Paiement (simulation)

Sécurité et gestion des accès

- Authentification sécurisée
- Rôles : Admin, Agent, Client

ESMIA

Projet 5 : Système de gestion d'un service de livraison

Présentation générale

Le projet vise à concevoir une application permettant de gérer efficacement un service de livraison, depuis la création des commandes jusqu'à leur livraison finale.

Le système doit permettre d'assigner automatiquement les livreurs, de suivre les livraisons en temps réel et de calculer les coûts en fonction de la distance parcourue, tout en optimisant les performances opérationnelles.

Objectifs du projet

- Automatiser la gestion des commandes de livraison.
- Optimiser l'attribution des livreurs pour réduire les délais.
- Assurer un suivi précis des livraisons.
- Calculer automatiquement les coûts de livraison.
- Fournir des indicateurs de performance (temps, efficacité, volume).

Fonctionnalités attendues

Création des commandes

- Création d'une commande avec :
  - Nom du client
  - Adresse de récupération (pickup)
  - Adresse de livraison
  - Type de colis (poids, taille)
  - Date et heure de livraison souhaitée
- Modification ou annulation de commande
- Attribution d'un statut initial : En attente

Attribution des livreurs

- Attribution automatique d'un livreur selon :
  - disponibilité
  - proximité géographique
  - charge de travail
- Possibilité d'attribution manuelle
- Réaffectation en cas d'imprévu

Suivi des livraisons

- Suivi en temps réel du statut :
  - En attente
  - Assignée

    - En cours
    - Livrée
    - Annulée
- Historique des livraisons
- Visualisation de la position du livreur (option GPS)

Logique métier (Business Logic)

Optimisation de l'attribution des livreurs

Le système doit attribuer le livreur le plus pertinent en fonction de plusieurs critères :

- distance entre le livreur et le point de récupération
- disponibilité du livreur
- nombre de livraisons en cours
- priorité de la commande

Calcul du prix selon la distance

Le coût de livraison doit être calculé automatiquement en fonction de :

- distance entre les points (pickup → destination)
- type de colis
- frais supplémentaires (urgent, fragile, etc.)

Suivi du statut des commandes

Chaque commande suit un cycle de vie strict :

(1)En attente → commande créée

(2)Assignée → livreur attribué

(3)En cours → livraison en route

(4)Livrée → livraison terminée

(5)Annulée → commande interrompue

Règles :

- Impossible de passer à "Livrée" sans être "En cours"
- Une commande annulée ne peut plus être modifiée
- Le statut doit être mis à jour automatiquement selon les actions

Contraintes techniques

- **Backend** : Java EE
- **Frontend** : Au choix, mais responsive
- **Base de données** : PostgreSQL ou MySQL
- **API REST**

Interfaces attendues

Interface Administrateur

- Gestion des commandes
- Gestion des livreurs
- Configuration des tarifs
- Suivi global des livraisons
- Statistiques

Interface Dispatcher (opérateur)

- Création des commandes
- Attribution manuelle
- Suivi en temps réel
- Gestion des incidents

Interface Livreur

- Réception des commandes
- Acceptation/refus
- Navigation vers destination
- Mise à jour du statut
- Historique des livraisons

Interface Client

- Création de commande
- Suivi en temps réel
- Historique
- Paiement (simulation)

Sécurité et gestion des accès

- Authentification sécurisée
- Rôles : Admin, Dispatcher, Livreur, Client

ESMIA